

cbuild

Felipe Cassio Rocha de Oliveira Lima¹, João Augusto Haupt Fonseca Oliveira¹,
Mateus Cruzatto Ramos¹, Sarah Antunes¹

¹Instituto de Matemática, Estatística e Ciência da Computação da Universidade de São Paulo (IME-USP) – Caixa Postal 05508-090 – São Paulo, SP – Brasil

{felipecassio, joao.haupt, mateus.cruzatto, sarahantunesvilela}@usp.br

Abstract. *Cbuild is a lightweight build tool for C projects written entirely in Bash. It scaffolds the project environment, compiles sources incrementally, runs the resulting program, cleans build artifacts, records its activity in a log file, and produces statistics and visual reports about the project.*

Resumo. *O cbuild é uma ferramenta leve de compilação de projetos em C, escrita inteiramente em Bash. Ele inicializa o ambiente do projeto, compila os fontes de forma incremental, executa o programa gerado, limpa arquivos temporários, registra suas atividades em um arquivo de log e produz estatísticas e relatórios visuais sobre o projeto.*

1. Introdução

O cbuild é uma pequena ferramenta de compilação para projetos em C, escrita inteiramente em Bash. Em vez de depender de sistemas como `make` ou `cmake`, ele reconstrói, em poucos scripts legíveis, o comportamento essencial de uma ferramenta de *build*: descoberta do projeto, configuração, compilação incremental e registro de atividades.

Suas principais funcionalidades são:

- criação do ambiente do projeto (diretórios, arquivo de configuração e arquivos-modelo);
- compilação incremental, que recompila apenas os fontes que mudaram;
- execução, limpeza e recompilação completa do projeto;
- exibição de informações e estatísticas do projeto, inclusive em um relatório HTML;
- arquivo de configuração que permite personalizar o nome do projeto, o nome do executável e o compilador;
- modo verboso, que exibe informações adicionais sobre as etapas executadas;
- modo *debug*, que auxilia o usuário a diagnosticar problemas;
- registro de todas as operações em um arquivo de log.

Este documento apresenta os requisitos e a instalação da ferramenta (Seção 2), sua interface (Seção 3), sua arquitetura (Seção 4) e os detalhes de implementação (Seções 5 a 8), encerrando com as limitações conhecidas e as considerações finais.

2. Requisitos e instalação

Para utilizar o cbuild são necessários:

- Bash;

- gcc ou clang;
- sed (GNU), tput e os utilitários usuais do sistema (find, grep, du, date e uname).

A instalação é feita pelo script `install.sh`, localizado na raiz do repositório. Ele torna o comando `cbuild` acessível de qualquer diretório, adicionando a pasta do projeto à variável de ambiente `PATH`:

```
git clone https://github.com/mat-afk/cbuild.git
cd cbuild
./install.sh
source ~/.bashrc
```

O script executa as seguintes etapas:

1. determina o caminho absoluto do diretório em que se encontra e o armazena em `CBUILD_ROOT`, de modo que funcione independentemente de onde o repositório foi clonado ou de onde o script é chamado;
2. procura esse caminho no arquivo `~/.bashrc`;
3. se o caminho ainda não estiver presente, acrescenta ao final do arquivo uma linha `export PATH` que inclui o diretório do `cbuild`; caso contrário, apenas informa que o `cbuild` já está no `PATH` e não altera o arquivo;
4. exporta o `PATH` atualizado para o próprio processo do script.

O trecho essencial do script (com as mensagens ao usuário omitidas) é:

```
1 CBUILD_ROOT="$ (cd _$(dirname "$0") _&&_pwd) "
2 BASHRC="$HOME/.bashrc"
3
4 if ! grep -qF "$CBUILD_ROOT" "$BASHRC"; then
5     echo "export PATH=\"\$PATH:$CBUILD_ROOT\" " >> "$BASHRC"
6 fi
7
8 export PATH="$PATH:$CBUILD_ROOT"
```

Como a verificação com `grep` precede a escrita, o script é *idempotente*: executá-lo várias vezes não cria entradas duplicadas. Como o script roda em um processo filho, a alteração do `PATH` só vale para novos terminais; por isso é necessário executar `source ~/.bashrc` ou abrir um novo terminal após a instalação.

Vale observar que o instalador atua apenas sobre o `~/.bashrc`. Usuários de outros *shells* devem adicionar a linha `export PATH` ao arquivo equivalente. Para desinstalar, basta remover essa linha do `~/.bashrc` e apagar o diretório do projeto.

3. Interface

A sintaxe geral de uso da ferramenta é:

```
cbuild <comando> [opcoes]
```

Os comandos disponíveis estão descritos na Tabela 1. Com exceção de `init` e `help`, todos devem ser executados dentro de um projeto `cbuild`, isto é, no diretório que contém o arquivo `cbuild.conf` ou em qualquer um de seus subdiretórios.

Tabela 1 – Comandos do cbuild

Comando	Descrição
init	Inicializa um novo projeto C no diretório atual, criando o arquivo de configuração <code>cbuild.conf</code> e a estrutura de diretórios.
build	Compila os fontes em arquivos-objeto e os liga no executável configurado. Apenas os arquivos que precisam ser recompilados são processados.
run	Compila (se o projeto não estiver atualizado) e executa o projeto.
clean	Remove os artefatos de compilação gerados pelo cbuild, sem afetar os fontes nem a configuração.
rebuild	Recompila o projeto do zero. Equivale a <code>clean</code> seguido de <code>build</code> .
info	Exibe informações sobre o projeto e sua configuração.
help	Exibe a mensagem de ajuda. Também é exibida com <code>-h</code> , <code>--help</code> ou diante de qualquer comando desconhecido.

Fonte: Os autores.

As opções, listadas na Tabela 2, devem ser informadas logo após o comando, como em `cbuild build --verbose`.

Tabela 2 – Opções do cbuild

Opção	Descrição
<code>-v, --verbose</code>	Modo verboso: exibe informações adicionais sobre o processo, incluindo os comandos executados pelo cbuild.
<code>-d, --debug</code>	Modo <i>debug</i> : exibe detalhes das operações internas da ferramenta.
<code>-h, --help</code>	Exibe a mensagem de ajuda.
<code>--html, --visual</code>	Usada com o comando <code>info</code> : gera uma página HTML com as estatísticas do projeto.

Fonte: Os autores.

Um fluxo típico de uso, do zero até a execução do programa, é:

```
mkdir hello && cd hello
cbuild init           # responde as perguntas (nome, binario,
                      # compilador)
cbuild build --verbose # compila exibindo as etapas
cbuild run            # executa o programa
cbuild info --html    # gera docs/info.html
cbuild clean          # remove os artefatos de compilacao
```

Com a opção `--verbose`, a compilação exibe as etapas em um estilo semelhante ao do `make`:

```
Building project...
```

```
CC main.c
LD build/obj/*.o app
```

4. Arquitetura

O `cbuild` é organizado como um despachante enxuto, que delega o trabalho a scripts pequenos e de propósito único. A estrutura do repositório é:

```
cbuild/
|-- cbuild
|-- install.sh
|-- cmd/
|   |-- init.sh
|   |-- build.sh
|   |-- run.sh
|   |-- clean.sh
|   |-- rebuild.sh
|   |-- info.sh
|   `-- help.sh
|-- lib/
|   |-- common.sh
|   |-- config.sh
|   `-- log.sh
|-- resources/
|   |-- cbuild.template.conf
|   |-- main.template.c
|   |-- main_test.template.c
|   `-- README.template.md
`-- example/
```

- **cbuild**: arquivo principal e ponto de entrada. Interpreta o comando e as opções informados pelo usuário e executa o script correspondente.
- **install.sh**: script de instalação, que adiciona o `cbuild` ao `PATH` (Seção 2).
- **cmd/**: um script para cada comando, contendo as funções e o código característico de cada um.
- **lib/**: código comum à ferramenta, utilizado em diferentes partes. O arquivo `common.sh` reúne funções auxiliares e o tratamento de erros, `config.sh` realiza a descoberta do projeto e o carregamento da configuração, e `log.sh` implementa o registro de atividades.
- **resources/**: arquivos-modelo utilizados pelo `init` na criação de novos projetos.
- **example/**: projeto de exemplo já compilado.

O fluxo de execução de uma chamada ao `cbuild` segue as etapas abaixo:

1. o arquivo `cbuild` determina o seu próprio diretório (`CBUILD_ROOT`) e carrega as bibliotecas `common.sh`, `config.sh` e `log.sh`;
2. se o comando exige um projeto (`build`, `run`, `clean`, `rebuild` e `info`), `config.sh` procura o `cbuild.conf`, define os caminhos do projeto e carrega a configuração; caso não o encontre, a execução é encerrada com um erro;

3. a primeira opção informada após o comando ativa o modo verboso ou o modo *debug*;
4. o script `cmd/<comando>.sh` correspondente é carregado com `source` e executado; comandos desconhecidos levam à tela de ajuda;
5. cada comando registra suas operações no arquivo `logs/cbuild.log`.

5. Descoberta da raiz do projeto

A função `find_project_root()`, definida em `lib/config.sh`, localiza o arquivo de configuração `cbuild.conf`. A busca começa no diretório atual e sobe pela hierarquia, examinando o diretório pai a cada iteração, até chegar à raiz do sistema (`/`). Ao encontrar o arquivo, a função imprime o diretório correspondente e retorna 0; se a busca terminar sem sucesso, retorna 1. Graças a essa estratégia, os comandos funcionam a partir de qualquer subdiretório do projeto.

O diretório encontrado passa a ser o `PROJECT_ROOT`, a partir do qual são definidas as variáveis `SRC_DIR`, `INCLUDE_DIR`, `BUILD_DIR`, `OBJ_DIR`, `DOCS_DIR`, `LOGS_DIR` e `BIN`. Em seguida, o arquivo `cbuild.conf` é carregado e o caminho do executável é ajustado conforme o nome configurado. A descoberta só é realizada para os comandos que dependem de um projeto:

```
1 find_project_root() {
2     local dir="$(pwd)"
3
4     while [[ "$dir" != "/" ]]; do
5         if [[ -f "$dir/cbuild.conf" ]]; then
6             echo "$dir"
7             return 0
8         fi
9
10        dir="$(dirname "$dir")"
11    done
12
13    return 1
14 }
15
16 if command_needs_conf "$cmd"; then
17     PROJECT_ROOT="$(find_project_root)" || throw_error
18     missing_conf
19
20     SRC_DIR="$PROJECT_ROOT/src"
21     INCLUDE_DIR="$PROJECT_ROOT/include"
22     BUILD_DIR="$PROJECT_ROOT/build"
23     OBJ_DIR="$BUILD_DIR/obj"
24     DOCS_DIR="$PROJECT_ROOT/docs"
25     LOGS_DIR="$PROJECT_ROOT/logs"
26
27     source "$PROJECT_ROOT/cbuild.conf"
28     BIN="$BUILD_DIR/$output"
29 fi
```

6. Implementação

Esta seção descreve brevemente a implementação de cada comando.

6.1. Ponto de entrada (**cbuild**)

O arquivo principal redireciona o comando do usuário ao script correspondente e verifica se o modo verboso e/ou o modo *debug* foram ativados com `-v` e `-d`. Ambos são representados por variáveis booleanas, inicialmente falsas:

```
1 flag="$1"
2 VERB=false
3 DBG=false
4
5 case "$flag" in
6     --verbose|-v)
7         VERB=true
8         ;;
9     --debug|-d)
10        DBG=true
11        ;;
12 esac
```

6.2. **init**

O comando verifica primeiro se o arquivo de configuração já existe; nesse caso, encerra com um erro. Caso contrário, solicita ao usuário, de forma interativa e com valores padrão, o nome do projeto (padrão: nome da pasta atual), o nome do binário (padrão: nome do projeto) e o compilador C, que pode ser `gcc` (padrão) ou `clang`. Em seguida, cria a estrutura de diretórios do projeto e a popula com os arquivos-modelo:

```
meu-projeto/
|-- cbuild.conf
|-- src/
|   '-- main.c
|-- include/
|-- build/
|   '-- obj/
|-- tests/
|   '-- main_test.c
|-- docs/
|   '-- README.md
'-- logs/
    '-- cbuild.log
```

Os arquivos `src/main.c` e `tests/main_test.c` só são copiados quando os respectivos diretórios estão vazios, de modo que o conteúdo já existente nunca é sobrescrito.

6.3. **build**

O comando busca, recursivamente, todos os arquivos `.c` do diretório `src`, compila cada um em um objeto de mesmo nome dentro de `build/obj` e liga todos os objetos

em um único executável. A compilação usa as opções `-Wall`, `-Wextra`, `-MMD` e `-MP` e inclui o diretório `include` no caminho de busca de cabeçalhos.

A compilação é **incremental**: um fonte só é recompilado se o seu objeto ainda não existir ou se o fonte for mais recente que o objeto. Ao final, são ligados tanto os objetos gerados na execução atual quanto os já compilados anteriormente.

```
1 compile() {
2     local source="$1"
3     local object="$2"
4
5     cmd=("$cc" -I$INCLUDE_DIR -Wall -Wextra -MMD -MP -c "$source
6         " -o "$object")
7     "${cmd[@]}"
8 }
9
10 build() {
11     local objects=()
12
13     mapfile -t sources < <(find "$SRC_DIR" -type f -name "*.c")
14
15     if [[ ${#sources[@]} == 0 ]]; then
16         throw_error sources_missing
17     fi
18
19     for source in "${sources[@]}; do
20         local filename="$(basename "$source" .c)"
21         local object="$OBJ_DIR/$filename.o"
22
23         objects+=("$object")
24
25         if [[ ! -f "$object" || "$source" -nt "$object" ]]; then
26             verbose "CC_$filename.c"
27             compile "$source" "$object"
28         fi
29     done
30
31     link "${objects[@]}"
32 }
```

6.4. run

Verifica se o projeto está atualizado e, quando necessário (por exemplo, se o executável não existir), aciona o `build` antes da execução. Se ainda assim o executável não existir, encerra com o erro `missing_binaries`. Por fim, executa o programa.

6.5. clean

Remove o executável e todos os arquivos `.o` e `.d` de `build/obj`. Os fontes, os cabeçalhos e o arquivo de configuração não são afetados.

6.6. rebuild

Executa, em sequência, as funções de `clean` e de `build`, resultando em uma compilação completa do projeto.

6.7. info

Exibe um resumo do projeto, organizado em três blocos: o sistema (versão do kernel e do compilador), as quantidades (tamanho do projeto, número de arquivos `.c` e `.h` e número de linhas) e as execuções (registros mais recentes de execução e de compilação, obtidos do arquivo de log). Com as opções `--html` ou `--visual`, o comando gera, em vez disso, o relatório visual descrito na Seção 7.

7. Funcionalidades adicionais

Aqui, vamos comentar as implementações de algumas funcionalidade adicionais do projeto.

7.1. Arquivo de configuração

O usuário pode escolher o nome do projeto, o nome do arquivo executável e o compilador. Essas escolhas são feitas durante o `init`, quando o usuário é consultado, e são gravadas no arquivo `cbuild.conf` pela função `scaffold_conf()`, definida em `cmd/init.sh`. Ela utiliza o `sed` para substituir os marcadores do modelo `resources/cbuild.template.conf`:

```
1 scaffold_conf() {
2     local target_dir="$1"
3     local project_name="$2"
4     local binary_name="$3"
5     local cc="$4"
6
7     sed \
8         -e "s|@@PROJECT@@|$project_name|g" \
9         -e "s|@@BINARY@@|$binary_name|g" \
10        -e "s|@@CC@@|$cc|g" \
11        "$RES_DIR/cbuild.template.conf" > "$target_dir/cbuild.
12        conf"
```

O modelo, cujos marcadores são substituídos, tem o seguinte conteúdo:

```
1 project="@@PROJECT@@ "
2 output="@@BINARY@@ "
3
4 cc="@@CC@@ "
```

Como o `cbuild.conf` é um arquivo Bash simples carregado com `source`, ele pode ser editado manualmente a qualquer momento.

7.2. Modos verboso e *debug*

A implementação dos modos está distribuída entre o arquivo `cbuild`, que ativa as variáveis `VERB` e `DBG` (Seção 6), e as bibliotecas `common.sh` e `log.sh`. A função `verbose()` imprime a mensagem apenas quando o modo verboso está ativo:

```
1 verbose() {
2     local message="$1"
3
4     if [[ $VERB == true ]]; then
5         echo "$message"
6     fi
7 }
```

As demais funções apenas chamam `verbose` com a mensagem desejada, como em `verbose "CC $filename.c"`, no `build`. Já o modo *debug* imprime no terminal as mensagens de nível `DEBUG` que, de qualquer forma, são sempre registradas no `log`.

7.3. Registro de atividades (log)

Todas as operações são registradas no arquivo `logs/cbuild.log`, uma entrada por linha, com as mais recentes no topo do arquivo. O formato de cada entrada é:

```
[Data Hora] [comando] [NIVEL] Mensagem
```

O *comando* identifica o script que gerou a entrada (por exemplo, `build`, `clean` ou `info`) e o nível pode ser `INFO`, `WARN`, `DEBUG` ou `ERROR`. Um exemplo de entradas reais é:

```
[2026-09-18 22:33:09] [run] [INFO] The project has started
running.
[2026-09-18 22:33:06] [build] [INFO] Project built successfully.
```

A função central de `lib/log.sh`, simplificada, é:

```
1 log() {
2     local level="$1"
3     local message="$2"
4     local command="${SCOPE:-"cbuild"}"
5
6     date="$(date_ '+%Y-%m-%d_%H:%M:%S' )"
7     log_entry="[$date]_[$command]_[$level]_$message"
8
9     if [[ -s "$LOG_FILE" ]]; then
10         sed -i "1i\\$log_entry" "$LOG_FILE"
11     else
12         echo "$log_entry" > "$LOG_FILE"
13     fi
14
15     if [[ $DBG == true && $level == "DEBUG" ]]; then
16         echo "DEBUG:_$message"
17     fi
18 }
```

7.4. Geração de relatórios

O comando `info`, implementado em `cmd/info.sh`, possui dois modos de saída, escolhidos pela opção informada. Sem opções, a função `default()` imprime o resumo no terminal. Com `--html` ou `--visual`, a função `visual()` gera o arquivo `docs/info.html`:

```
1 case "$flag" in
2     --html|--visual)
3         visual
4         log_info "Info_visual_created."
5         ;;
6     *)
7         default
8         log_info "Info_created."
9         ;;
10 esac
```

O relatório HTML apresenta cartões com as informações do sistema (kernel e compilador) e do projeto (tamanho, linhas, arquivos `.c` e arquivos de cabeçalho) e um gráfico de pizza com a proporção entre arquivos `.c` e `.h`, desenhado com a biblioteca Plotly. Como essa biblioteca é carregada a partir de uma CDN, a visualização do gráfico requer conexão com a internet.

8. Tratamento de erros

Optamos por centralizar o tratamento de erros em `lib/common.sh`, o que simplifica a manutenção e garante mensagens consistentes. Cada situação de erro é identificada por um nome e tratada pela função `throw_error()`, que seleciona a mensagem apropriada e chama, ao final, a função `die()`:

```
1 die() {
2     local message="$1"
3     echo "$message" 1>&2 ; exit 1
4 }
```

Essa função imprime a mensagem na saída de erro padrão e encerra o processo com código de saída diferente de zero. Os erros contemplados estão resumidos na Tabela 3.

Tabela 3 – Erros tratados pelo `cbuild`

Identificador	Situação
<code>missing_conf</code>	Comando executado fora de um projeto <code>cbuild</code> .
<code>already_initialized</code>	<code>init</code> em um diretório que já contém <code>cbuild.conf</code> .
<code>missing_cc</code>	Compilador configurado não está instalado.
<code>invalid_cc</code>	Compilador não reconhecido (diferente de <code>gcc</code> e <code>clang</code>).
<code>missing_directory</code>	Diretório necessário ao projeto não existe.
<code>sources_missing</code>	Nenhum arquivo <code>.c</code> em <code>src</code> .
<code>missing_binaries</code>	<code>run</code> sem executável disponível.
<code>permission_denied</code>	Executável sem permissão de execução.

Fonte: Os autores.

9. Limitações e trabalhos futuros

Algumas limitações da versão atual do `cbuild`, que indicam caminhos para evolução da ferramenta, são:

- as opções são reconhecidas apenas quando informadas imediatamente após o comando (por exemplo, `cbuild build -v`, mas não `cbuild -v build`);
- a decisão de recompilar compara apenas cada fonte com o seu objeto; os arquivos de dependência (`.d`) são gerados, mas ainda não são usados para detectar alterações em cabeçalhos;
- os objetos recebem o nome-base do fonte, de modo que dois fontes com o mesmo nome em subdiretórios distintos colidiriam;
- o diretório `tests` é criado, com um modelo de teste, mas os testes ainda não são compilados nem executados pelo `cbuild`;
- apenas os compiladores `gcc` e `clang` são suportados;
- o instalador atua somente sobre o `~/ .bashrc`.

10. Considerações finais

Em relação às dificuldades encontradas, houve um certo estranhamento com a linguagem e suas convenções, especialmente devido à pouca similaridade com outras linguagens mais comuns, como C e Python. Por outro lado, o desenvolvimento mostrou que é possível reproduzir, com poucos scripts, o comportamento essencial de uma ferramenta de *build*. Para o futuro, acreditamos que uma maior modularização e organização do projeto ajudaria imensamente durante o desenvolvimento, e que as limitações listadas acima são um ponto de partida natural para as próximas etapas.